

how our model is performing on the training dataset. Keras provides us with a number of metrics such as `binary_accuracy`, `categorical_accuracy`, `sparse_categorical_accuracy`, `top_k_categorical_accuracy`, and `sparse_top_k_categorical_accuracy`. Since we are performing a two-class classification problem; the accuracy metric is a good choice. The choice of the metric depends on the type of task that we are doing.

The next important step before we start training our model is to perform some Normalization. It is the process of scaling our image pixel values to have a standard deviation of 1 and a mean of 0.

We will perform some important functions like normalisation when we are feeding images into our model during training using an important module in keras called `ImageDataGenerator`.

ImageDataGenerator library:

According to the creator of keras François Chollet, `ImageDataGenerator` (.) library of Keras enables us to quickly set-up python generators that automatically turn image files into pre - processed tensors. These tensors can be fed directly into models during training. It performs the following functions:



Figure : Using ImageDataGenerator library

1. Decode the JPEG content of the images to Red Green Blue (RGB) grids of pixels of 3 channels.
2. Convert these three RGB channels into floating point tensors.
3. Normalise data by rescaling the pixel values that were between 0 and 255 to the [0, 1] interval, with 0 standards mean and one as standard deviation.
4. Since we are training on a small data set, it is necessary to augment images. Keras `ImageDataGenerator()` helps us to augment images easily.

Using Image Data Generator library and training our model:

Frist create our `ImageDataGenerator` object. We will need to create two